

JAVA SPEZIAL

Vergleichen und Sortieren

Alles, was Sie für die Prüfung wissen müssen

Natürliche Ordnung, Comparable, Comparator,
Arrays.sort, Collections.sort, binarySearch

In diesem Skript:

- Warum Ordnung wichtig ist: Sortieren und Suchen
- Natürliche Ordnung mit Comparable
- Flexible Ordnungen mit Comparator
- Sortieren von Arrays und Listen (Arrays.sort, Collections.sort)
- Suchen mit Arrays.binarySearch
- Mehrstufiges Sortieren (z. B. Person nach Name, dann Alter)
- Typische Stolperfallen und Best Practices

Autor: Benjamain Malicke ©

Datum: 23. März 2026

Inhaltsverzeichnis

1	Vergleichen, Sortieren und Suchen	2
1.1	Sortieren mit Arrays.sort und Collections.sort	2
1.2	Suchen mit Arrays.binarySearch	3
1.3	Brücke zu Comparable und Comparator	4
2	Natürliche Ordnung mit Comparable	5
2.1	Beispielklasse Person mit natürlicher Ordnung nach Name . .	5
2.2	Verwendung von Person in Arrays und Listen	6
2.3	Natürliche Ordnung: Welches Kriterium ist sinnvoll?	7
3	Flexible Ordnungen mit Comparator	8
3.1	Eigene Comparator-Klassen für Person	9
3.2	Anonyme Klassen und Lambdas als Comparator	10
3.3	Mehrstufige Sortierung: Name, dann Alter	11
4	Mehrstufige Sortierung per String-Konkatenation (und ihre Tücken)	13
4.1	Grundidee: Alter + Name konkatenieren	13
4.2	Problem 1: Lexikografische vs. numerische Ordnung (2 vs. 10)	14
4.3	Lösungsidee: Zahlen mit führenden Nullen formatieren	14
4.4	Problem 2: Trennzeichen und Sonderfälle	15
4.5	Saubere Alternative: Direkter Vergleich der Felder	16
5	binarySearch mit Person-Arrays	16
5.1	binarySearch mit der natürlichen Ordnung (Comparable) . .	17
5.2	binarySearch mit Comparator	18
6	Zusammenfassung: Comparable, Comparator und binarySearch	19
A	Schnellübersicht für die Prüfung	23
	Anhang A: Schnellübersicht für die Prüfung	23
B	Übungsaufgaben	24
	Anhang B: Übungsaufgaben	24
	Lösungen zu Anhang B	27

1 Vergleichen, Sortieren und Suchen

In vielen Programmen müssen Daten nicht nur gespeichert, sondern auch **geordnet** und **gezielt durchsucht** werden. Typische Beispiele sind:

- eine Liste von Namen in alphabetischer Reihenfolge,
- eine Übersicht von Produkten nach Preis oder Bewertung,
- Datensätze, die nach Datum sortiert und durchsucht werden.

Damit das effizient funktioniert, braucht Java eine **definierte Ordnung** der Elemente. Für Basisdatentypen (z. B. `int`, `double`) und für viele Klassen der Standardbibliothek (z. B. `String`) ist diese Reihenfolge bereits festgelegt. Für eigene Klassen (z. B. `Person`) müssen wir sie selbst bestimmen.

Merkkasten

Eine sinnvolle Ordnung der Elemente ist die Grundlage für

- **Sortieralgorithmen** (z.B. `Arrays.sort`, `Collections.sort`) und
- effiziente **Suchalgorithmen** wie die binäre Suche (`binarySearch`).

Ohne klar definierte Reihenfolge gibt es keine definierte Sortierung – und effizientes Suchen wird schwierig.

1.1 Sortieren mit `Arrays.sort` und `Collections.sort`

Für Arrays und Listen stellt Java bereits fertige Methoden zum Sortieren bereit:

- `Arrays.sort(...)` für Arrays,
- `Collections.sort(...)` oder `List.sort(...)` für Listen.

Listing 1: Sortieren eines int-Arrays und einer String-Liste

```
1  import java.util.*;
2
3  public class SortBeispiel {
4      public static void main(String[] args) {
5          int[] zahlen = { 5, 2, 9, 1, 5, 6 };
6          Arrays.sort(zahlen); // sortiert aufsteigend
7
8          System.out.println("Sortiertes Array: " + Arrays.
9                               toString(zahlen));
10
11         List<String> namen = new ArrayList<>();
12         namen.add("Anna");
13         namen.add("Peter");
14         namen.add("Chris");
15
16         Collections.sort(namen); // alphabetisch (natü
17                                     rliche Ordnung von String)
18         // alternativ: namen.sort(null);
19
20         System.out.println("Sortierte Liste: " + namen);
21     }
22 }
```

Merkkasten

Arrays.sort und **Collections.sort** verwenden die **natürliche Ordnung** der Elemente, wenn keine andere Sortierung angegeben wird. Beispiele für natürliche Ordnung:

- Zahlen: aufsteigend nach ihrem Wert,
- String: lexikographisch (ungefähr „alphabetisch“),
- eigene Klassen: die Ordnung, die wir mit **Comparable** festlegen.

1.2 Suchen mit **Arrays.binarySearch**

Ist ein Array **sortiert**, kann man es viel schneller durchsuchen als mit einer einfachen Schleife. Java stellt dafür die Methode **Arrays.binarySearch(...)** zur Verfügung. Sie setzt die **binäre Suche** ein:

- beginnt in der **Mitte** des Bereichs,
- entscheidet anhand des Vergleichs, ob links oder rechts weitergesucht wird,
- teilt den Suchbereich in jedem Schritt ungefähr in zwei Hälften.

Dadurch wächst die Anzahl der Vergleiche nur logarithmisch mit der Array-Größe ($O(\log n)$) und nicht linear ($O(n)$).

Listing 2: Suchen mit `Arrays.binarySearch`

```
1  import java.util.*;
2
3  public class BinarySearchBeispiel {
4      public static void main(String[] args) {
5          int[] zahlen = { 1, 2, 5, 5, 6, 9 };
6
7          int index = Arrays.binarySearch(zahlen, 5);
8          System.out.println("Index von 5: " + index);
9
10         int indexNichtGefunden = Arrays.binarySearch(zahlen
11             , 7);
12         System.out.println("Suchergebnis für 7: " +
13             indexNichtGefunden);
14     }
15 }
```

Stolperfallen

Sortiertheit ist Pflicht!

`Arrays.binarySearch` setzt voraus, dass das Array nach **derselben Ordnung** sortiert ist, nach der auch gesucht wird.

- Wird ein unsortiertes Array mit `binarySearch` durchsucht, sind die Ergebnisse **nicht definiert** (falsche oder scheinbar zufällige Indizes).
- Auch bei eigenen Klassen muss die Reihenfolge beim Sortieren und beim Suchen zusammenpassen.

Verwenden Sie `binarySearch` daher nur bei Arrays, die vorher korrekt sortiert wurden (z. B. mit `Arrays.sort`).

1.3 Brücke zu `Comparable` und `Comparator`

Bisher haben wir mit Typen gearbeitet, deren Ordnung Java bereits "kennt" (z. B. `int`, `String`). In der Praxis arbeiten wir jedoch oft mit **eigenen Klassen**, etwa `Person` mit Attributen wie Name und Alter.

Damit Java auch solche Objekte sinnvoll sortieren und suchen kann, müssen wir festlegen:

- Was bedeutet „kleiner“, „größer“ oder „gleich“ für unsere Klasse?
- Nach welchem Kriterium (Name, Alter, Gehalt, ...) sollen Objekte standardmäßig geordnet werden?

Genau hier kommen die Schnittstellen **Comparable** und **Comparator** ins Spiel:

- **Comparable** legt die **natürliche Ordnung** in der Klasse selbst fest.
- **Comparator** erlaubt **beliebige alternative Ordnungen** von außen (z. B. nach Alter statt nach Name).

Im nächsten Abschnitt definieren wir eine Klasse **Person** und statten sie mit einer natürlichen Ordnung aus, indem wir **Comparable<Person>** implementieren.

2 Natürliche Ordnung mit Comparable

Die Schnittstelle **Comparable<T>** legt eine **natürliche Ordnung** für Objekte einer Klasse fest.

Ein Objekt kann sich dann selbst mit einem anderen Objekt gleichen Typs vergleichen.

Die zentrale Methode ist:

```
int compareTo(T other)
```

Sie gibt

- einen **negativen** Wert zurück, wenn **this < other**,
- **0**, wenn **this == other** (im Sinne der Ordnung),
- einen **positiven** Wert, wenn **this > other**.

Merkkasten

Comparable bestimmt die Standard-Reihenfolge einer Klasse. Überall dort, wo Java eine natürliche Ordnung braucht (z. B. `Arrays.sort`, `Collections.sort` ohne `Comparator`, `TreeSet`, `TreeMap`), wird die `compareTo`-Methode verwendet.

2.1 Beispielklasse Person mit natürlicher Ordnung nach Name

Wir betrachten eine Klasse **Person** mit den Attributen **name** und **alter**. Die natürliche Ordnung soll alphabetisch nach dem Namen erfolgen.

Listing 3: Klasse Person mit Comparable-Implementierung

```
1 public class Person implements Comparable<Person> {
2     private String name;
3     private int alter;
4
5     public Person(String name, int alter) {
6         this.name = name;
7         this.alter = alter;
8     }
9
10    public String getName() {
11        return name;
12    }
13
14    public int getAlter() {
15        return alter;
16    }
17
18    @Override
19    public String toString() {
20        return name + " (" + alter + ")";
21    }
22
23    @Override
24    public int compareTo(Person other) {
25        // Natürliche Ordnung: alphabetisch nach name
26        return this.name.compareTo(other.name);
27    }
28 }
```

Merkkasten

Vergleich von Strings

String implementiert selbst Comparable<String>. Daher können wir einfach `name.compareTo(other.name)` verwenden.

2.2 Verwendung von Person in Arrays und Listen

Sobald `Person` `Comparable<Person>` implementiert, können wir Arrays und Listen von `Person` ohne zusätzlichen Comparator sortieren. Java verwendet automatisch die implementierte natürliche Ordnung (hier: Name).

Listing 4: Sortieren eines Person-Arrays und einer Liste

```
1  import java.util.*;
2
3  public class PersonDemo {
4      public static void main(String[] args) {
5          Person[] personenArray = {
6              new Person("Zoe", 19),
7              new Person("Anna", 22),
8              new Person("Peter", 20)
9          };
10
11         Arrays.sort(personenArray); // nutzt Person.
                                     compareTo
12         System.out.println("Sortiertes Array: " + Arrays.
                               toString(personenArray));
13
14         List<Person> personenListe = new ArrayList<>();
15         personenListe.add(new Person("Chris", 25));
16         personenListe.add(new Person("Bernd", 30));
17         personenListe.add(new Person("Anna", 22));
18
19         Collections.sort(personenListe); // nutzt ebenfalls
                                     compareTo
20         System.out.println("Sortierte Liste: " +
                               personenListe);
21     }
22 }
```

Stolperfallen

Inkonsistente compareTo-Implementierung

Die compareTo-Methode sollte folgende Eigenschaften erfüllen:

- **Antisymmetrie:** $\text{sgn}(a.\text{compareTo}(b)) = -\text{sgn}(b.\text{compareTo}(a))$,
- **Transitivität:** Wenn $a < b$ und $b < c$, dann sollte $a < c$ gelten,
- **Konsistenz mit equals (empfohlen):** Wenn $a.\text{compareTo}(b) == 0$, dann sollte meist auch $a.\text{equals}(b) == \text{true}$ gelten.

Werden diese Regeln verletzt, kann das zu unerwartetem Verhalten in sortierten Collections (z. B. `TreeSet`, `TreeMap`) führen.

2.3 Natürliche Ordnung: Welches Kriterium ist sinnvoll?

Bei echten Projekten muss man sich gut überlegen, welches Kriterium die natürliche Ordnung einer Klasse bestimmen soll:

- **Person:** nach Name? Nach Nachname, dann Vorname? Nach eindeutiger ID?
- **Produkt:** nach Artikelnummer? Nach Preis (aufsteigend oder absteigend)?

Die natürliche Ordnung sollte intuitiv und stabil sein. Für alternative Sortierkriterien (z.B. Person nach Alter statt nach Name) verwenden wir später **Comparator**en.

Im nächsten Abschnitt erweitern wir das Beispiel: Wir bleiben bei der Klasse **Person**, betrachten aber weitere Sortierkriterien und kombinieren sie (z.B. zuerst nach Name, bei gleichem Namen nach Alter).

3 Flexible Ordnungen mit Comparator

Mit **Comparable** legen wir genau *eine* natürliche Ordnung in der Klasse selbst fest. Oft wollen wir aber dieselbe Klasse nach **verschiedenen Kriterien** sortieren:

- **Person** nach Name,
- oder nach Alter,
- oder nach mehreren Kriterien (zuerst Name, dann Alter),
- oder sogar absteigend nach einem bestimmten Wert.

Dafür bietet Java die Schnittstelle **Comparator<T>** an. Ein **Comparator** ist ein eigenständiges Objekt, das zwei Elemente miteinander vergleichen kann.

Die zentrale Methode lautet (in ihrer einfachsten Form):

```
int compare(T a, T b)
```

Sie erfüllt dieselben Rückgaberegeln wie **compareTo**:

- negativer Wert: $a < b$,
- 0: $a == b$ (in Bezug auf diese Ordnung),
- positiver Wert: $a > b$.

Merkkasten

Comparatoren erlauben mehrere alternative Ordnungen.

Comparable definiert die Standard-Reihenfolge *in der Klasse*. **Comparator**en definieren flexible Ordnungen *von außen*.

3.1 Eigene Comparator-Klassen für Person

Wir bleiben bei der Klasse `Person` aus Section 2. Dort war die natürliche Ordnung nach dem Namen definiert.

Nun erstellen wir zusätzliche Comparatoren:

- **nach Alter** (aufsteigend),
- **nach Alter** (absteigend),
- **zuerst nach Name, dann nach Alter** (mehrstufige Sortierung).

Zunächst ein klassischer Comparator als eigene Klasse:

Listing 5: Comparator fuer Person nach Alter (aufsteigend)

```
1 import java.util.Comparator;
2
3 public class PersonAlterComparator implements
4     Comparator<Person> {
5     @Override
6     public int compare(Person p1, Person p2) {
7         // aufsteigende Sortierung nach Alter
8         return Integer.compare(p1.getAlter(), p2.getAlter())
9     }
10 }
```

Merkkasten

Integer.compare statt p1.getAlter() - p2.getAlter()

Verwenden Sie bei Ganzzahlen am besten `Integer.compare(a, b)`. Die Variante `a - b` kann bei sehr großen Zahlen theoretisch zu Überläufen führen und ist schlechter lesbar.

Diesen Comparator können wir nun beim Sortieren angeben:

Listing 6: Verwendung eines Comparators beim Sortieren

```
1  import java.util.*;
2
3  public class PersonComparatorDemo {
4      public static void main(String[] args) {
5          List<Person> personen = new ArrayList<>();
6          personen.add(new Person("Zoe", 19));
7          personen.add(new Person("Anna", 22));
8          personen.add(new Person("Peter", 20));
9          personen.add(new Person("Anna", 30));
10
11         // Natuerliche Ordnung (Name) - Comparable
12         Collections.sort(personen);
13         System.out.println("Sortiert nach Name (Comparable)
14                             : " + personen);
15
16         // Alternative Ordnung: nach Alter (Comparator)
17         Collections.sort(personen, new
18             PersonAlterComparator());
19         System.out.println("Sortiert nach Alter (Comparator
20                             ): " + personen);
21     }
22 }
```

3.2 Anonyme Klassen und Lambdas als Comparator

Seit Java 8 lassen sich Comparatoren sehr bequem mit **Lambdas** und `Comparator.comparing(...)` definieren. Für viele Fälle brauchen wir keine eigene Klasse mehr.

Listing 7: Comparatoren mit Lambda-Ausdruecken

```
1  import java.util.*;
2
3  public class PersonLambdaDemo {
4      public static void main(String[] args) {
5          List<Person> personen = List.of(
6              new Person("Zoe", 19),
7              new Person("Anna", 22),
8              new Person("Peter", 20),
9              new Person("Anna", 30)
10         );
11
12         List<Person> liste = new ArrayList<>(personen);
13
14         // Sortierung nach Alter mit Lambda
15         liste.sort((p1, p2) -> Integer.compare(p1.getAlter(), p2.getAlter()));
16         System.out.println("Nach Alter (Lambda): " + liste);
17
18         // Dasselbe mit Comparator.comparing
19         liste = new ArrayList<>(personen);
20         liste.sort(Comparator.comparing(Person::getAlter));
21         System.out.println("Nach Alter (comparing): " +
22             liste);
23     }
24 }
```

Merkkasten

Comparator.comparing vereinfacht viele Comparatoren.
Statt

```
(p1, p2) -> Integer.compare(p1.getAlter(), p2.getAlter())
```

reicht oft

```
Comparator.comparing(Person::getAlter)
```

3.3 Mehrstufige Sortierung: Name, dann Alter

Häufig reicht ein einziges Kriterium nicht aus. Beispiel: Personen sollen

1. zuerst alphabetisch nach Name sortiert werden,
2. bei gleichem Namen nach Alter aufsteigend.

Ohne Hilfsmethoden könnten wir das direkt im Comparator ausdrücken:

Listing 8: Mehrstufiger Vergleich: Name, dann Alter

```
1 import java.util.Comparator;
2
3 public class PersonNameDannAlterComparator implements
4     Comparator<Person> {
5     @Override
6     public int compare(Person p1, Person p2) {
7         int nameVergleich = p1.getName().compareTo(p2.
8             getName());
9         if (nameVergleich != 0) {
10             return nameVergleich; // Unterschiedlicher Name
11                 entscheidet
12         }
13         // Namen sind gleich -> nach Alter vergleichen
14         return Integer.compare(p1.getAlter(), p2.getAlter()
15             );
16     }
17 }
```

Mit Java 8 und Comparator-Hilfsmethoden geht das sehr elegant:

Listing 9: Mehrstufige Sortierung mit thenComparing

```
1 import java.util.*;
2
3 public class PersonMehrstufigDemo {
4     public static void main(String[] args) {
5         List<Person> personen = new ArrayList<>();
6         personen.add(new Person("Anna", 30));
7         personen.add(new Person("Zoe", 19));
8         personen.add(new Person("Anna", 22));
9         personen.add(new Person("Peter", 20));
10
11         // Zuerst nach Name, dann nach Alter
12         Comparator<Person> comp = Comparator
13             .comparing(Person::getName)
14             .thenComparing(Person::getAlter);
15
16         personen.sort(comp);
17
18         System.out.println("Sortiert nach Name, dann Alter:
19             " + personen);
20     }
21 }
```

Merkkasten

thenComparing fuer zweite (und dritte,...) Sortierstufe

Mit `thenComparing` lassen sich leicht weitere Kriterien anhaengen:

```
Comparator.comparing(Person::getName)
    .thenComparing(Person::getAlter)
    .thenComparing(Person::getGehalt);
```

Die Reihenfolge der Aufrufe entspricht der Sortierreihenfolge.

Stolperfallen

Comparator muss zur verwendeten Struktur passen

- In `TreeSet` und `TreeMap` entscheidet der `Comparator` (oder die natuerliche Ordnung), wann ein Element als „gleich“ gilt. Wenn `compare(a, b) == 0`, wird kein weiteres Element eingefuegt.
- Unterschiedliche Comparatoren koennen zu unterschiedlicher Vorstellung von Gleichheit fuehren als `equals()`.

Planen Sie Comparatoren daher sorgfaeltig, vor allem wenn sie in sortierten Collections eingesetzt werden.

4 Mehrstufige Sortierung per String-Konkatenation (und ihre Tücken)

In seltenen Fällen wird versucht, eine mehrstufige Sortierung zu erreichen, indem mehrere Attribute zu einem einzigen `String` verknüpft und dann `String.compareTo` verwendet werden. Die Idee:

- mehrere Felder (z. B. Alter und Name) werden zu einem Schlüssel zusammengesetzt, z. B. „42:Anna“ oder „19:Peter“,
- die Sortierung erfolgt dann über diesen zusammengesetzten String.

4.1 Grundidee: Alter + Name konkatenieren

Beispiel: Wir möchten Personen zuerst nach Alter, dann nach Name sortieren und benutzen dafür einen zusammengesetzten Schlüssel-String.

Listing 10: Mehrstufiger Vergleich per Konkatination

```
1  @Override
2  public int compareTo(Person other) {
3      String thisKey = this.alter + ":" + this.name;
4      String otherKey = other.alter + ":" + other.name;
5
6      return thisKey.compareTo(otherKey);
7  }
```

Merkkasten

Idee: Die Reihenfolge der Teile im Schlüssel (zuerst `alter`, dann `name`) bestimmt auch die Sortierreihenfolge: Es wird **zuerst nach Alter**, bei gleichem Alter dann nach Name sortiert.

4.2 Problem 1: Lexikografische vs. numerische Ordnung (2 vs. 10)

`String.compareTo` vergleicht Zeichenketten **lexikografisch**, nicht numerisch. Das führt bei Zahlen schnell zu falscher Reihenfolge.

Beispiel-Alterswerte als Strings:

- lexikografisch: "2" < "10" ist **falsch**, denn das erste Zeichen von "10" ist '1', das von "2" ist '2' – also gilt lexikografisch: "10" < "2".
- numerisch wollen wir aber: $2 < 10$.

Übertragen auf den zusammengesetzten Schlüssel:

- "2:Anna" und "10:Bernd"
- lexikografische Ordnung: "10:Bernd" kommt **vor** "2:Anna", weil '1' < '2'.
- numerisch korrekt wäre aber: $2 < 10$, also "2:Anna" zuerst.

Stolperfallen

Wer numerische Werte einfach als Text vergleicht, bekommt oft eine **falsche Reihenfolge**. Bei einstelligen und zweistelligen Zahlen fällt das sehr schnell auf (Beispiel 2 vs. 10).

4.3 Lösungsidee: Zahlen mit führenden Nullen formatieren

Man kann den Fehler abschwächen, indem man Zahlen vor der Konkatination auf eine feste Breite mit führenden Nullen formatiert:

Listing 11: Konkatenation mit fuehrenden Nullen

```
1  @Override
2  public int compareTo(Person other) {
3      // Alter auf 3 Stellen mit fuehrenden Nullen
4      // formatieren, z.B. 2 -> "002"
5      String thisKey = String.format("%03d", this.alter) +
6      ":" + this.name;
7      String otherKey = String.format("%03d", other.alter)
8      + ":" + other.name;
9
10     return thisKey.compareTo(otherKey);
11 }
```

Damit sind folgende Schlüssel lexikografisch **korrekt** geordnet:

- "002:Anna"
- "010:Bernd"
- "120:Chris"

Denn jetzt gilt lexikografisch: "002« "010« "120", was der numerischen Ordnung $2 < 10 < 120$ entspricht.

Merkkasten

Durch führende Nullen kann die lexikografische Ordnung von Zahlen an die numerische Ordnung angeglichen werden. Trotzdem bleibt der Ansatz **unnötig kompliziert** und fehleranfällig.

4.4 Problem 2: Trennzeichen und Sonderfälle

Weitere Schwierigkeiten bei der Konkatenationsmethode:

- Sie brauchen ein **Trennzeichen** (z. B. ":"), das garantiert nicht in den einzelnen Attributen vorkommt – sonst kann man die Teile nicht eindeutig auseinanderhalten.
- Wenn später weitere Kriterien hinzukommen (z. B. Gehalt), muss das Format des Schlüssels sehr sorgfältig angepasst werden.
- Bei komplexen Typen (Datumsangaben, IDs, optionale Felder) wird der zusammengesetzte String schnell **unübersichtlich** und fehleranfällig.

4.5 Saubere Alternative: Direkter Vergleich der Felder

Statt komplizierter Texte ist es in der Praxis viel klarer, die Felder direkt zu vergleichen:

Listing 12: Sauberer mehrstufiger Vergleich ohne Strings

```
1  @Override
2  public int compareTo(Person other) {
3      // 1. Kriterium: Alter (numerisch korrekt)
4      int cmp = Integer.compare(this.alter, other.alter);
5      if (cmp != 0) {
6          return cmp;
7      }
8
9      // 2. Kriterium: Name (lexikografisch)
10     return this.name.compareTo(other.name);
11 }
```

Oder mit einem Comparator:

Listing 13: Mehrstufiger Comparator mit thenComparing

```
1  Comparator<Person> comp = Comparator
2  .comparingInt(Person::getAlter) // 1. Kriterium
3  .thenComparing(Person::getName); // 2. Kriterium
4
5  personen.sort(comp);
```

Merkkasten

Empfehlung: Für echte Projekte und in Prüfungen ist der **direkte Vergleich der Felder** (mit `Integer.compare`, `Comparator.comparing`, `thenComparing`) die deutlich saubere, lesbarere und weniger fehleranfällige Variante. Die String-Konkatenation ist eher ein Notbehelf und sollte höchstens als Lernbeispiel gezeigt werden.

In den folgenden Abschnitten koennen wir nun gezielt zeigen, wie `Comparable` und `Comparator` mit `Arrays.sort`, `Collections.sort` und `Arrays.binarySearch` zusammenspielen (z. B. binäre Suche auf sortierten `Person`-Arrays).

5 binarySearch mit Person-Arrays

Die Methode `Arrays.binarySearch` kann nicht nur mit primitiven Typen und `String`, sondern auch mit eigenen Klassen wie `Person` verwendet werden. **Voraussetzung** ist immer:

- das Array ist nach einer bestimmten Ordnung sortiert, und

- `binarySearch` verwendet *dieselbe* Ordnung (`Comparable` oder denselben `Comparator`).

5.1 `binarySearch` mit der natürlichen Ordnung (`Comparable`)

Wenn `Person` (wie in Section 2) `Comparable<Person>` implementiert und die natürliche Ordnung nach `name` definiert ist, können wir `binarySearch` direkt auf einem sortierten `Person[]` verwenden.

Listing 14: `Comparable`] `binarySearch` mit `Person[]` und `Comparable`

```

1  import java.util.Arrays;
2
3  public class PersonBinarySearchDemo {
4      public static void main(String[] args) {
5          Person[] personen = {
6              new Person("Anna", 22),
7              new Person("Bernd", 30),
8              new Person("Chris", 25),
9              new Person("Peter", 20)
10         };
11
12         // 1. Sortieren nach der natuerlichen Ordnung (Name
13         // )
14         Arrays.sort(personen); // nutzt Person.compareTo
15         System.out.println("Sortiertes Array: " + Arrays.
16             toString(personen));
17
18         // 2. Suchobjekt erzeugen
19         Person suchPerson = new Person("Chris", 0);
20         // Alter ist hier egal, wenn compareTo nur den
21         // Namen verwendet
22
23         int index = Arrays.binarySearch(personen,
24             suchPerson);
25         System.out.println("Index von " + suchPerson.
26             getName() + ": " + index);
27
28         if (index >= 0) {
29             System.out.println("Gefundenes Objekt: " +
30                 personen[index]);
31         } else {
32             System.out.println("Nicht gefunden.
33                 Einfuegeposition: " + (-index - 1));
34         }
35     }
36 }

```

Merkkasten

Suchobjekt und Vergleichskriterium müssen zusammenpassen

- Im Beispiel vergleicht `compareTo` nur den `name`.
- Deshalb reicht es, beim Suchobjekt einen passenden Namen zu setzen; andere Felder (z. B. `alter`) können beliebig sein.
- Würde `compareTo` auch das Alter berücksichtigen, müsste das Suchobjekt *alle* relevanten Felder korrekt gesetzt haben.

5.2 binarySearch mit Comparator

Alternativ kann `binarySearch` auch mit einem `Comparator` arbeiten. In diesem Fall gilt:

- Das Array muss **mit demselben Comparator** sortiert worden sein,
- `binarySearch` bekommt denselben `Comparator` übergeben.

Listing 15: `und Comparator]binarySearch` mit `Person[]` und `Comparator`

```
1  import java.util.Arrays;
2  import java.util.Comparator;
3
4  public class PersonBinarySearchMitComparator {
5      public static void main(String[] args) {
6          Person[] personen = {
7              new Person("Anna", 30),
8              new Person("Zoe", 19),
9              new Person("Anna", 22),
10             new Person("Peter", 20)
11         };
12
13         // Comparator: zuerst nach Name, dann nach Alter
14         Comparator<Person> comp = Comparator
15             .comparing(Person::getName)
16             .thenComparing(Person::getAlter);
17
18         // 1. Sortieren mit diesem Comparator
19         Arrays.sort(personen, comp);
20         System.out.println("Sortiert (Name, dann Alter): "
21             + Arrays.toString(personen));
22
23         // 2. Suchobjekt passend zum Comparator bauen
24         Person suchPerson = new Person("Anna", 22);
25
26         int index = Arrays.binarySearch(personen,
27             suchPerson, comp);
```

```

27     System.out.println("Index von " + suchPerson + ": " +
28         + index);
29
30     if (index >= 0) {
31         System.out.println("Gefundenes Objekt: " +
32             personen[index]);
33     } else {
34         System.out.println("Nicht gefunden.
35             Einfuegeposition: " + (-index - 1));
36     }
37 }
38 }
39 }

```

Stolperfallen

Sortierung und Suche müssen denselben Comparator verwenden

- Wird das Array mit einem anderen Comparator sortiert als dem, der in `binarySearch` verwendet wird, sind die Ergebnisse **nicht definiert**.
- Ebenso falsch ist es, ein Array mit `Comparable` (natürliche Ordnung) zu sortieren, aber mit einem Comparator zu durchsuchen (oder umgekehrt).

Achten Sie darauf, dass Sortierung und Suche immer dieselbe Ordnung nutzen.

6 Zusammenfassung: Comparable, Comparator und binarySearch

In diesem Skript haben wir gesehen, wie Java Objekte vergleicht, sortiert und effizient durchsucht. Die wichtigsten Punkte im Überblick:

Comparable vs. Comparator (Überblick)

Comparable<T>	Definiert die natürliche Ordnung einer Klasse direkt in der Klasse selbst (<code>compareTo</code>). Wird verwendet, wenn keine andere Ordnung angegeben wird.
Comparator<T>	Definiert alternative Ordnungen von außen (<code>compare(a, b)</code>). Ermöglicht mehrere verschiedene Sortierkriterien für dieselbe Klasse.

Merkkasten

Kurz gesagt:

- Comparable: „Wie soll diese Klasse standardmäßig sortiert werden?“
- Comparator: „Welche andere Sortierung brauche ich gerade?“

Wichtige Methoden und Idiome

- **Natürliche Ordnung:**

- `class Person implements Comparable<Person>`
 `... int compareTo(Person other)...`
- In `compareTo`: negatives Ergebnis = `this < other`, 0 = gleich, positiv = `this > other`.

- **Comparatoren:**

- Klassische Implementierung: `class X implements Comparator<Person>`
- Moderne Schreibweise (Java 8+):

```
Comparator.comparing(Person::getName)
            .thenComparing(Person::getAlter);
```

- **Sortieren:**

- `Arrays.sort(array)` – nutzt `Comparable`.
- `Arrays.sort(array, comp)` – nutzt angegebenen `Comparator`.
- `Collections.sort(list)` oder `list.sort(null)` – nutzt `Comparable`.
- `list.sort(comp)` – nutzt angegebenen `Comparator`.

- **Suchen mit `binarySearch`:**

- `Arrays.binarySearch(array, key)` – Array muss nach der natürlichen Ordnung sortiert sein.
- `Arrays.binarySearch(array, key, comp)` – Array muss mit *demselden* `Comparator` sortiert sein.

Typische Stolperfallen

Stolperfallen

1. Inkonsistente compareTo- oder compare-Methoden

- Verletzung von Transitivität und Konsistenz kann zu schwer nachvollziehbaren Fehlern in `TreeSet`, `TreeMap` und beim Sortieren führen.
- Faustregel: Wenn `a.compareTo(b) == 0`, sollte meist auch `a.equals(b)` gelten.

Stolperfallen

2. Sortierung und `binarySearch` passen nicht zusammen

- `binarySearch` setzt konsequent sortierte Arrays voraus.
- Sortierung und Suche müssen **dieselbe Ordnung** verwenden (entweder beide `Comparable` oder beide denselben `Comparator`).

Stolperfallen

3. Auswahl der natürlichen Ordnung

- Eine unpassende natürliche Ordnung kann zu Verwirrung führen (z.B. Person nach interner ID statt nach Name).
- Wichtige, intuitive Ordnung als `Comparable` wählen, alle anderen Sortierungen über `Comparatoren` abbilden.

Merkkasten für die Prüfung

Merkkasten

Für die Prüfung merken:

- `Comparable<T>`: Methode `int compareTo(T other)`.
- `Comparator<T>`: Methode `int compare(T a, T b)`.
- `Arrays.sort` / `Collections.sort`: nutzen `Comparable` oder übergebenen `Comparator`.
- `Arrays.binarySearch`: nur auf passend sortierten Arrays verwenden.
- Mehrstufige Sortierung: `Comparator.comparing(...).thenComparing(...)`.

A Schnellübersicht für die Prüfung

Vergleichen in Java – Kurzüberblick

Comparable<T>

- In der Klasse selbst implementiert: `class Person implements Comparable<Person>`
- Methode: `int compareTo(T other)`
- Legt die **natürliche Ordnung** fest (Standard-Sortierung).

Comparator<T>

- Externes Objekt, das zwei Werte vergleicht.
- Methode: `int compare(T a, T b)`
- Ermöglicht **alternative Ordnungen** (z.B. nach Alter statt Name).

Typische Muster

Natürliche Ordnung (Comparable)

```
1 public class Person implements Comparable<Person> {
2     private String name;
3     private int alter;
4
5     @Override
6     public int compareTo(Person other) {
7         // Beispiel: nach Name sortieren
8         return this.name.compareTo(other.name);
9     }
10 }
```

Comparator – alternative Ordnung

```
1 Comparator<Person> nachAlter =
2     Comparator.comparingInt(Person::getAlter);
3
4 Comparator<Person> nachNameDannAlter =
5     Comparator.comparing(Person::getName)
6     .thenComparingInt(Person::getAlter);
```


Sortieren und Suchen

Sortieren

- Array mit natürlicher Ordnung: `Arrays.sort(array);`
- Array mit Comparator: `Arrays.sort(array, comp);`
- Liste: `Collections.sort(list);` oder `list.sort(comp);`

binarySearch

- `Arrays.binarySearch(array, key)` → Array muss nach natürlicher Ordnung sortiert sein.
- `Arrays.binarySearch(array, key, comp)` → Array muss mit demselben Comparator sortiert sein.
- Rückgabe ≥ 0 : Index gefunden, < 0 : „Einfügeposition“.

Stolperfallen – ganz kurz

- **Sortierung und Suche müssen zusammenpassen:** gleicher Comparator bzw. gleiche natürliche Ordnung.
- **Inkonsistente compareTo/compare-Logik** kann TreeSet/TreeMap kaputt machen.
- Zahlen nicht als String vergleichen („10“ vs. „2“), sondern `Integer.compare(...)` bzw. `Comparator.comparingInt(...)` nutzen.

B Übungsaufgaben

Hinweis

Alle Aufgaben beziehen sich auf die Klasse `Person` wie im Skript (`name`, `alter`, ggf. `gehalt`).

Aufgabe 1: Natürliche Ordnung definieren

Gegeben ist eine einfache Klasse `Person`:

```

1  public class Person {
2      private String name;
3      private int alter;
4
5      public Person(String name, int alter) {
6          this.name = name;
7          this.alter = alter;
8      }
9
10     public String getName() { return name; }
11     public int getAlter() { return alter; }
12 }

```

Aufgabe:

1. Erweitern Sie **Person**, so dass die Klasse **Comparable<Person>** implementiert.
2. Definieren Sie die natürliche Ordnung so, dass zuerst nach **name** (alphabetisch) sortiert wird.

Aufgabe 2: Comparatoren für verschiedene Sortierungen

Aufgabe:

1. Schreiben Sie einen Comparator, der Personen nach **Alter** aufsteigend sortiert.
2. Schreiben Sie einen Comparator, der nach **Gehalt** absteigend sortiert (falls Sie kein Gehalt im Modell haben, wählen Sie ein anderes numerisches Feld).
3. Erstellen Sie eine Liste von mindestens 4 Personen und sortieren Sie diese einmal nach natürlicher Ordnung und einmal nach dem Comparator aus Teil (a).

Aufgabe 3: Mehrstufige Sortierung

Aufgabe:

1. Definieren Sie eine Sortierung: zuerst nach **name** (alphabetisch), bei gleichem Namen nach **alter** aufsteigend.
2. Lösen Sie dies einmal mit einem **eigenen Comparator** (compare-Methode mit **if**) und einmal mit **Comparator-Hilfsmethoden** (**Comparator.comparing**, **thenComparing**).

Aufgabe 4: binarySearch mit Person[]

Gegeben sei ein Array von Personen:

```
1 Person[] personen = {  
2     new Person("Anna", 22),  
3     new Person("Bernd", 30),  
4     new Person("Chris", 25),  
5     new Person("Peter", 20)  
6 };
```

Aufgabe:

1. Sortieren Sie das Array nach der natürlichen Ordnung von `Person` (aus Aufgabe 1).
2. Suchen Sie mit `Arrays.binarySearch` eine `Person` mit dem Namen „Chris“.
3. Erklären Sie kurz, warum das Suchobjekt bestimmte Felder korrekt gesetzt haben muss (Stichwort: Vergleichskriterium in `compareTo`).

Aufgabe 5: String-Konkatenation (Fehler finden)

Betrachten Sie folgende (absichtlich naive) Implementierung einer `compareTo`-Methode:

```
1 @Override  
2 public int compareTo(Person other) {  
3     String thisKey = this.alter + ":" + this.name;  
4     String otherKey = other.alter + ":" + other.name;  
5     return thisKey.compareTo(otherKey);  
6 }
```

Aufgabe:

1. Erklären Sie, warum diese Implementierung bei den Alterswerten 2 und 10 zu einer falschen Reihenfolge führen kann.
2. Schlagen Sie eine saubere Alternative vor, bei der die Felder direkt verglichen werden.

Lösungen zu Anhang B

Lösung zu Aufgabe 1

```
1 public class Person implements Comparable<Person> {
2     private String name;
3     private int alter;
4
5     public Person(String name, int alter) {
6         this.name = name;
7         this.alter = alter;
8     }
9
10    public String getName() { return name; }
11    public int getAlter() { return alter; }
12
13    @Override
14    public int compareTo(Person other) {
15        // Natuerliche Ordnung: alphabetisch nach name
16        return this.name.compareTo(other.name);
17    }
18 }
```

Lösung zu Aufgabe 2

Comparator nach Alter (aufsteigend):

```
1 Comparator<Person> nachAlter =
2     Comparator.comparingInt(Person::getAlter);
```

Comparator nach Gehalt (absteigend) – Beispiel:

```
1 Comparator<Person> nachGehaltAbsteigend =
2     Comparator.comparingDouble(Person::getGehalt)
3     .reversed();
```

Verwendung:

```
1 List<Person> personen = new ArrayList<>();
2 personen.add(new Person("Zoe", 19));
3 personen.add(new Person("Anna", 22));
4 personen.add(new Person("Peter", 20));
5
6 // Natuerliche Ordnung (Name)
7 Collections.sort(personen);
8
9 // Alternative Ordnung: nach Alter
10 personen.sort(nachAlter);
```

Lösung zu Aufgabe 3

Variante mit eigener Comparator-Klasse oder anonymer Klasse:

```
1  Comparator<Person> nameDannAlter = new Comparator<Person>() {
2      @Override
3      public int compare(Person p1, Person p2) {
4          int cmp = p1.getName().compareTo(p2.getName());
5          if (cmp != 0) {
6              return cmp; // Name entscheidet
7          }
8          // Namen gleich -> nach Alter vergleichen
9          return Integer.compare(p1.getAlter(), p2.getAlter());
10     }
11 };
```

Variante mit Comparator-Hilfsmethoden:

```
1  Comparator<Person> nameDannAlter2 =
2      Comparator.comparing(Person::getName)
3      .thenComparingInt(Person::getAlter);
```

Lösung zu Aufgabe 4

(a) Sortieren nach natürlicher Ordnung:

```
1  Arrays.sort(personen); // nutzt Person.compareTo (nach Name)
```

(b) Suchen mit `binarySearch`:

```
1  Person suchPerson = new Person("Chris", 0);
2  int index = Arrays.binarySearch(personen, suchPerson);
```

(c) Begründung:

Wenn `compareTo` nur den `name` vergleicht, reicht es, beim Suchobjekt den Namen korrekt zu setzen; das Alter ist dann für den Vergleich egal. Würde `compareTo` auch das Alter berücksichtigen, müssten *alle* relevanten Felder im Suchobjekt passend gesetzt sein, damit `binarySearch` das Element findet.

Lösung zu Aufgabe 5

(a) Problem 2 vs. 10:

Die Implementierung baut Schlüssel wie "2:Anna" und "10:Bernd" und vergleicht sie mit `String.compareTo`. Dieser Vergleich ist **lexikografisch**: Er schaut Zeichen für Zeichen.

Dabei gilt: "10" < "2", weil das erste Zeichen '1' kleiner ist als '2'. Dadurch kann z.B. "10:Bernd" vor "2:Anna" einsortiert werden, obwohl numerisch $2 < 10$ gilt.

(b) Saubere Alternative:

```
1  @Override
2  public int compareTo(Person other) {
3      // 1. Kriterium: Alter (numerisch korrekt)
4      int cmp = Integer.compare(this.alter, other.alter);
5      if (cmp != 0) {
6          return cmp;
7      }
8      // 2. Kriterium: Name (lexikografisch)
9      return this.name.compareTo(other.name);
10 }
```

Oder mit Comparator:

```
1  Comparator<Person> comp =
2  Comparator.comparingInt(Person::getAlter)
3  .thenComparing(Person::getName);
```